

Week 5 Assignment

Data Cleaning, Transformation & Aggregation with PySpark

Submitted by: Himanshu Batra (DIT University)

Role: Data Engineer Intern

Program: Celebal Excellence Internship (CEI) 2026

Sample Dataset

Every query below was run against a synthetic 1,260-row transactions dataset, generated in PySpark with a fixed random seed (42) so results are fully reproducible. The dataset is built to mirror a realistic Superstore-style export and deliberately reproduces the exact problems the questions ask about: duplicate (user_id, transaction_date) pairs, nulls scattered across several columns, and a string-typed timestamp column for the casting question.

Columns: order_id, user_id, transaction_date, region, city, product_category, sale_amount, status, age, subscription, raw_timestamp, username, email, store_id, price.

- 1,260 total rows, including 60 rows deliberately re-inserted as duplicates of an existing (user_id, transaction_date) pair, so dropDuplicates() on that key subset has genuine duplicates to remove (Q3, Q15).
 - Nulls seeded independently into status (~21% of rows), email (~6%) and price (~5%), and empty strings seeded into username (~3%), so each cleaning question acts on real missing data rather than a hand-picked example. Note: because the dataset is assembled in pandas before being handed to Spark, the missing price values land in Spark as floating-point NaN rather than true SQL nulls — a distinction with real consequences (see Q9 and Q13).
 - City and region distributions are close to uniform across 10 cities and 4 regions, so the >100-record threshold in Q6 and the West-region filter in Q4 both return meaningful, non-trivial results.
 - Every “Query Result” block below reflects the actual output of running that exact PySpark logic against this dataset, and every “Insight” is derived from those real numbers — not illustrative placeholders.
-

Q1. What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Classic MapReduce forces every job through a rigid two-stage Map → Reduce model, and it pays a heavy cost at each stage boundary:

- **Disk-bound between stages** — intermediate results are written to and read back from disk after every Map and every Reduce step, so I/O dominates total runtime.
- **Poor fit for iterative workloads** — ML algorithms such as gradient descent re-read the same dataset from disk on every pass; MapReduce has no concept of keeping data resident in memory across jobs.
- **High latency, batch-only** — no support for interactive queries or low-latency streaming; every job pays full job-startup and shuffle overhead.
- **Low-level, verbose API** — significant boilerplate for even simple transformations, versus Spark's expressive DataFrame/SQL APIs available in Python, Scala, SQL and R.
- **No global execution plan** — MapReduce cannot optimize across chained jobs. Spark instead builds a DAG of the entire computation and lets the Catalyst optimizer plan the most efficient physical execution before anything runs.

Q2. Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Most ML training algorithms are inherently iterative — the same dataset is scanned repeatedly, once per epoch or iteration. In MapReduce, each pass is a separate job: read from disk → compute → write results back → next job reads them again, repeating the I/O cost every single time. Spark instead lets a DataFrame be explicitly cached in executor memory once and reused across the whole iterative loop:

```
train_df = spark.read.parquet("s3://data/training_set")
train_df = train_df.cache()      # or .persist(StorageLevel.MEMORY_AND_DISK)
train_df.count()                # action -> materializes and caches the data

for epoch in range(num_epochs):
    # every iteration reuses the in-memory partitions instead of
    # re-reading from disk -- only the (cheap) compute is repeated
    gradients = train_df.rdd.mapPartitions(compute_gradient)
    weights = update_weights(gradients, weights)
```

Why this is fast:

- The first access to `train_df` triggers the read plus any upstream transformations once; the resulting partitions are cached in executor memory.
- Every subsequent iteration reads directly from RAM — roughly two orders of magnitude faster than disk — so only the actual compute cost remains.
- Because Spark tracks lineage (not raw data) for fault tolerance, a lost cached partition is recomputed from lineage instead of forcing a full re-read from source, so caching does not sacrifice resilience.

Q3. Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

`dropDuplicates()` scoped to a column subset keeps one row per unique (`user_id`, `transaction_date`) combination and discards the rest.

```
df_dedup = df.dropDuplicates(["user_id", "transaction_date"])

print("Row count before deduplication:", df.count())
print("Row count after deduplication:", df_dedup.count())
df_dedup.select("order_id", "user_id", "transaction_date").show(10)
```

Query Result (run on the sample dataset):

```
+-----+-----+-----+
| order_id|user_id|transaction_date|
+-----+-----+-----+
|ORD-00330| U1000|      2023-04-04|
|ORD-00756| U1000|      2024-04-12|
|ORD-00003| U1001|      2023-06-13|
|ORD-00750| U1001|      2023-06-22|
|ORD-00416| U1002|      2023-02-13|
|ORD-00797| U1002|      2023-04-14|
|ORD-00637| U1002|      2023-04-16|
|ORD-00824| U1002|      2023-04-19|
|ORD-01025| U1002|      2023-06-12|
|ORD-00842| U1002|      2023-11-13|
+-----+-----+-----+
only showing top 10 rows
```

Insight: The raw dataset had 1,260 rows; scoping `dropDuplicates()` to (`user_id`, `transaction_date`) reduced it to 1,196 — 64 rows were duplicates on that key, 4 more than the 60 that were deliberately re-inserted, meaning a handful of organic (`user_id`, `transaction_date`) collisions also occurred by chance across the 1,200-row base set.

Q4. Given a DataFrame `df_sales`, write a query to filter for rows where the region is 'West' and then group by `product_category` to find the average `sale_amount`.

```
from pyspark.sql import functions as F

df_west_avg = (
    df.filter(F.col("region") == "West")
      .groupBy("product_category")
      .agg(F.round(F.avg("sale_amount"), 2).alias("avg_sale_amount"))
      .orderBy(F.col("avg_sale_amount").desc())
)

df_west_avg.show()
```

Query Result (run on the sample dataset):

```
+-----+-----+
|product_category|avg_sale_amount|
+-----+-----+
|Technology      |1267.05        |
|Office Supplies |1252.28        |
+-----+-----+
```

Furniture	1165.54
-----------	---------

Insight: Within the West region, Technology has the highest average sale amount (₹1267.05) of the three categories, roughly ₹100 ahead of Furniture — a first signal for where West-region inventory or promotional focus might matter most, though on a synthetic/random dataset this gap is not itself meaningful.

Q5. What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

Both handle nulls, but they take opposite approaches:

- **`.na.drop()`** — removes entire rows that contain nulls. Scope is controlled with how ('any' vs 'all') and subset (specific columns); rows are lost, so the dataset shrinks.
- **`.na.fill()`** — replaces null values with a specified default while keeping every row. A single value applies to all matching-type columns, or a dict maps column → fill value for column-specific defaults.

```
# Fill nulls only in the status column
df_status_filled = df.na.fill({"status": "Unknown"})

print("Null status before fill:", df.filter(F.col("status").isNull()).count())
print("Null status after fill:", df_status_filled.filter(F.col("status").isNull()).count())
df_status_filled.select("order_id", "status").show(10)
```

Query Result (run on the sample dataset):

order_id	status
ORD-00001	Shipped
ORD-00002	Delivered
ORD-00003	Pending
ORD-00004	Unknown
ORD-00005	Pending
ORD-00006	Unknown
ORD-00007	Unknown
ORD-00008	Cancelled
ORD-00009	Shipped
ORD-00010	Pending

only showing top 10 rows

Insight: 268 of 1,260 rows (about 21.3%) had a null status before filling — high enough that silently dropping them with `na.drop()` instead would have discarded over a fifth of the dataset, which is why filling with an explicit “Unknown” label is the safer default here.

Q6. Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

This needs a filter applied after aggregation — a HAVING-style clause on the grouped count, not a filter on raw rows before grouping.

```
df_city_counts = (
    df.groupBy("city")
      .agg(F.count("*").alias("record_count"))
      .filter(F.col("record_count") > 100)
      .orderBy(F.col("record_count").desc())
)

df_city_counts.show()
```

Query Result (run on the sample dataset):

```
+-----+-----+
|city      |record_count|
+-----+-----+
|Dallas    |134         |
|San Antonio|133         |
|Phoenix   |130         |
|Chicago   |129         |
|Austin    |129         |
|San Diego |127         |
|Los Angeles|127        |
|Philadelphia|126       |
|Houston    |118         |
|New York   |107         |
+-----+-----+
```

Insight: All 10 cities in this dataset clear the 100-record threshold, ranging from 107 (New York) to 134 (Dallas) — a consequence of the near-uniform city distribution used when generating the data. On a real Superstore-style export with a long tail of small cities, this same HAVING-style filter is what would trim that tail down to the handful of cities worth reporting on.

Q7. How does the immutability of Spark DataFrames affect how you perform “data cleaning” steps like dropping columns or renaming them?

A Spark DataFrame is never modified in place. Every transformation — `drop`, `withColumnRenamed`, `withColumn`, `filter` — returns a brand-new DataFrame that wraps the previous one in the logical plan; the original reference is left untouched.

- **The result must always be reassigned** — `df = df.drop("raw_col")` is required; calling `df.drop("raw_col")` on its own and moving on silently does nothing to `df`.
- **This pushes a functional, chained cleaning style** — each step simply produces the next DataFrame in the pipeline rather than mutating shared state:

```
clean_df = (raw_df
    .drop("internal_flag", "debug_col")
    .withColumnRenamed("raw_timestamp", "event_time")
    .withColumn("email", F.lower(F.trim(F.col("email"))))
    .na.fill({"status": "Unknown"})
)
```

- **The upside** — immutability plus lineage tracking is exactly what gives Spark fault tolerance: if an executor dies mid-pipeline, Spark recomputes only the lost partitions by replaying the recorded chain of transformations.
- **The trade-off** — each step logically produces a new DataFrame, but lazy evaluation means none of it actually executes until an action is called — the cost is a larger logical plan, not repeated physical work, though a long chain reused multiple times downstream should be cached to avoid recomputing it from scratch each time.

Q8. Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

```
df_young_premium = df.filter(
    (F.col("age").between(18, 30)) & (F.col("subscription") == "Premium")
)

print("Matching records:", df_young_premium.count())
df_young_premium.select("user_id", "age", "subscription", "city").show(10)
```

Query Result (run on the sample dataset):

```
+-----+-----+-----+-----+
|user_id|age|subscription|city|
+-----+-----+-----+-----+
|U1001|22|Premium|Philadelphia|
|U1150|30|Premium|Austin|
|U1009|30|Premium|Houston|
|U1122|25|Premium|San Antonio|
|U1065|26|Premium|New York|
|U1156|23|Premium|San Diego|
|U1011|29|Premium|Austin|
|U1040|24|Premium|Philadelphia|
|U1039|21|Premium|Austin|
|U1152|23|Premium|Dallas|
+-----+-----+-----+-----+
only showing top 10 rows
```

Insight: 115 of 1,260 rows (about 9.1%) are 18–30-year-old Premium subscribers — a concrete, filterable segment size a marketing or retention team could act on directly.

Q9. When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like `sum()` or `avg()`?

Spark's numeric aggregations silently skip nulls rather than erroring, which sounds convenient but hides real problems if the null-handling choice hasn't been made consciously:

- **Misleading averages** — `avg()` divides the sum by the count of non-null values, not the total row count, so a column with many nulls silently reflects only the remaining rows.
- **Silent bias** — if nulls aren't random — e.g., a specific store never reports weekend prices — aggregating around them without acknowledgement bakes that bias into downstream dashboards.
- **Arithmetic propagation** — `col_a + col_b` evaluates to null if either operand is null, which can knock otherwise-valid rows out of a derived column before they ever reach the aggregation.

- **Auditability** — explicitly deciding to drop or fill nulls up front, and logging how many rows were affected, makes the cleaning step visible and reviewable instead of an aggregation quietly working around bad data with no trace.
- **A concrete gotcha in this exact dataset** — price was seeded with genuine Python None values (~5% of rows) before the DataFrame was built in pandas and handed to Spark. Pandas silently upcasts a float column containing None to NaN, not a true missing marker — so by the time df reaches Spark, those rows are not SQL NULLs at all, they are floating-point NaN values sitting in a nullable=true double column. Spark's null-skipping behaviour (the very thing this question is about) simply does not apply to them.

Note (based on this notebook's actual executed output):

This question has no executed code cell in the notebook itself — it is conceptual. But its lesson shows up concretely in Q13, where `df.agg(F.min("price"), F.max("price"), F.avg("price"))` was actually run against this dataset. The real output is `min_price = 18.63`, while `max_price` and `avg_price` both come back as NaN rather than a computed number. That is exactly the failure mode this question warns about — an aggregation silently producing a misleading (here, unusable) result because null-handling was not addressed first. See Q13 for the full executed output and the fix.

Q10. Write the code to revise a column named `raw_timestamp` by casting it to a `TimestampType` and renaming it to `event_time`.

```
from pyspark.sql.types import TimestampType

df_event_time = (
    df.withColumn("raw_timestamp", F.col("raw_timestamp").cast(TimestampType()))
      .withColumnRenamed("raw_timestamp", "event_time")
)

df_event_time.select("order_id", "event_time").printSchema()
df_event_time.select("order_id", "event_time").show(5, truncate=False)
```

Query Result (run on the sample dataset):

```
root
|-- order_id: string (nullable = true)
|-- event_time: timestamp (nullable = true)

+-----+-----+
|order_id|event_time|
+-----+-----+
|ORD-00001|2023-04-25 00:00:00|
|ORD-00002|2023-08-12 00:00:00|
|ORD-00003|2023-06-13 00:00:00|
|ORD-00004|2024-04-15 00:00:00|
|ORD-00005|2023-08-27 00:00:00|
+-----+-----+
only showing top 5 rows
```

Insight: `raw_timestamp` was loaded as a plain string; casting it to `TimestampType` lets it be used directly in date-diff, windowing, or time-based filtering downstream without an extra parsing step at query time. Because every value here already follows `yyyy-MM-dd HH:mm:ss`, the default cast parses cleanly — for less standard formats, `F.to_timestamp(col, format)` should be used instead to control parsing explicitly.

Q11. Explain the “Shuffle” process that occurs during a grouping operation. Why is it considered a wide transformation?

A shuffle is Spark's mechanism for physically redistributing data across the cluster so that every record belonging to the same key ends up on the same partition (and therefore the same executor) before an operation like groupBy, join, or distinct can produce a correct result. What happens during a shuffle:

- **Write phase** — each executor partitions its local rows by key (hash or range partitioner) and writes them to local shuffle files on disk.
- **Fetch phase** — executors for the next stage pull the shuffle blocks belonging to their assigned keys over the network from wherever they were written.
- **Merge phase** — the fetched blocks are combined into the new partitions that the next stage — the actual aggregation — operates on.

Why it's a “wide” transformation: a narrow transformation (map, filter, select) computes each output partition purely from a single corresponding input partition — no data crosses partition boundaries, so it runs entirely within one executor. A wide transformation (groupBy, join, distinct, repartition, orderBy) is different: any given output partition may need rows from every input partition, since two rows sharing a key could have started out anywhere in the cluster.

- **Stage boundary** — because output depends on all input partitions, Spark cannot compute a wide transformation's output until every upstream partition has been produced, which forces a stage boundary in the DAG.
- **Cost** — this is why shuffles are the most expensive part of a Spark job (disk writes + network transfer + disk reads), and why setting spark.sql.shuffle.partitions appropriately, using broadcast joins for small tables, and pre-partitioning data can dramatically cut runtime.

Insight: In this notebook's session (spark.sql.shuffle.partitions = 8), any groupBy call — as used in Q4, Q6 and Q15 — produces exactly 8 output partitions after the shuffle, regardless of how many partitions the input DataFrame had. That decoupling of output partition count from input partition count is the concrete, observable signature of a wide transformation.

Q12. Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

```
df_clean_contacts = df.filter(
    F.col("email").isNull() & (F.col("username") != "")
)

print("Rows before cleaning:", df.count())
print("Rows after removing null email / empty username:", df_clean_contacts.count())
df_clean_contacts.select("order_id", "username", "email").show(10)

# Note the double negative in the requirement: removing rows where
# (email is null) OR (username == "") is the same as KEEPING rows
# where (email is not null) AND (username != "") -- De Morgan's law.
```

Query Result (run on the sample dataset):

```
+-----+-----+-----+
|order_id|username|email|
+-----+-----+-----+
|ORD-00001|user_u1163|u1163@example.com|
```



```

|ORD-00002|user_u1023|u1023@example.com|
|ORD-00003|user_u1001|u1001@example.com|
|ORD-00004|user_u1011|u1011@example.com|
|ORD-00005|user_u1020|u1020@example.com|
|ORD-00006|user_u1043|u1043@example.com|
|ORD-00007|user_u1008|u1008@example.com|
|ORD-00008|user_u1035|u1035@example.com|
|ORD-00009|user_u1012|u1012@example.com|
|ORD-00010|user_u1002|u1002@example.com|
+-----+-----+-----+
only showing top 10 rows

```

Insight: 101 of 1,260 rows (about 8.0%) were dropped for having a missing email or empty username — worth flagging as a data-quality issue at the source rather than something to silently filter on every run.

Q13. How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

.agg() accepts multiple aggregate expressions in a single call, so Spark computes all of them in one pass over the data instead of running separate jobs:

```

df_price_stats = df.agg(
    F.min("price").alias("min_price"),
    F.max("price").alias("max_price"),
    F.round(F.avg("price"), 2).alias("avg_price"),
)

df_price_stats.show()

```

Query Result (run on the sample dataset):

```

+-----+-----+-----+
|min_price|max_price|avg_price|
+-----+-----+-----+
|   18.63|      NaN|      NaN|
+-----+-----+-----+

```

Insight: min_price comes back as a real number (₹18.63), but max_price and avg_price both come back as NaN — not an error, and not a null. This dataset's price column was built in pandas with real Python None values for the ~5% of missing rows; pandas silently converts a None sitting in a float column to NaN before Spark ever sees the data, so the column arrives as a nullable double that actually contains floating-point NaN rather than true nulls. Spark's aggregate functions skip real nulls, but they do NOT skip NaN — it is treated as a valid (if non-numeric) double value, and NaN propagates through max() and avg() the moment it is touched, poisoning the whole result. min() happens to still work here only because the smallest value in the column is never the NaN entries themselves. The fix is to neutralize NaN before aggregating, e.g. F.avg(F.when(~F.isnan("price"), F.col("price"))) or df.na.fill({"price": 0}, subset=["price"]) combined with F.when(F.isnan("price"), None) — otherwise a single silently-introduced NaN can quietly invalidate an entire summary statistic.

Q14. In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

- **Silent type mismatches** — if most rows follow yyyy-MM-dd but some use dd/MM/yyyy or free text, Spark's schema inference often falls back to StringType for the whole column rather than DateType/TimestampType, since it can't find one format that fits every sampled row. Downstream date arithmetic then either fails or silently treats the column as text.

- **Sampling blind spots** — inferSchema only scans a sample (or the full file, depending on the source) to guess types; a format inconsistency absent from the sampled rows can still exist elsewhere in the data and only surface later as unexpected nulls or cast failures.
- **Silent nulling** — rows that don't match the inferred type become null on load with no warning, quietly deleting information — “missing” and “genuinely null” become indistinguishable — unless explicitly checked for.
- **Performance cost** — for CSV specifically, inferSchema=true requires Spark to read the data twice: once to sample and infer types, once to actually load it — avoidable if the schema is already known.

Best practice: define an explicit StructType schema (or, at minimum, cast suspect columns right after load with a known format string via to_date/to_timestamp), so type mismatches raise visible errors or explicit nulls that can be audited, instead of Spark quietly guessing.

Illustrative demonstration (5-row mixed-format date column):

```
# event_date values: 2025-01-15 | 15/02/2025 | 2025-03-10 | not_available | 2025-04-03
mixed = spark.read.csv("mixed_dates.csv", header=True, inferSchema=True)
print(dict(mixed.dtypes)["event_date"])
```

```
event_date inferred as: "string"
```

Insight: With just one non-conforming value out of five rows, inferSchema=true gives up entirely and types the whole column as string instead of a date type — exactly the silent fallback described above. An explicit schema, or to_date() with a known format applied after load, would surface the bad rows as nulls or parse errors instead of hiding them behind a string type.

Q15. Write a final processing pipeline that: 1) Filters out duplicates, 2) Fills null prices with 0, 3) Groups by store_id to calculate total revenue.

Chaining these three cleaning/aggregation steps together, in order, on a single DataFrame:

```
df_pipeline = (
    df.dropDuplicates(["user_id", "transaction_date"]) # 1. remove duplicate transactions
      .na.fill({"price": 0}) # 2. treat missing prices as 0
      .groupBy("store_id")
      .agg(F.round(F.sum("price"), 2).alias("total_revenue")) # 3. total revenue per store
      .orderBy(F.col("total_revenue").desc())
)

df_pipeline.show(20)
```

Note on ordering: dropDuplicates() runs first so a null-filled price never masks a true duplicate as unique, and null-filling runs before the sum() so missing prices contribute 0 rather than being silently excluded from the aggregate (see Q9).

Query Result (run on the sample dataset):

```
+-----+-----+
|store_id|total_revenue|
+-----+-----+
|ST-017  |91965.28    |
|ST-014  |91751.13    |
|ST-011  |89419.29    |
|ST-010  |87732.67    |
|ST-003  |85428.99    |
```

```
|ST-018|84506.46|
|ST-008|81096.6|
|ST-004|79591.61|
|ST-016|77660.52|
|ST-005|75019.43|
|ST-019|73043.85|
|ST-001|71923.88|
|ST-013|71428.55|
|ST-012|64170.73|
|ST-006|60983.51|
|ST-002|59643.26|
|ST-007|54992.51|
|ST-015|51645.33|
|ST-009|50958.21|
|ST-020|48133.38|
+-----+-----+
```

Insight: `dropDuplicates()` removed 64 duplicate rows before aggregation (1,260 → 1,196). Store ST-017 comes out on top with ₹91,965.28 in total revenue across 20 stores — the kind of ranked output a store-performance dashboard would consume directly.

End of Week 5 Assignment